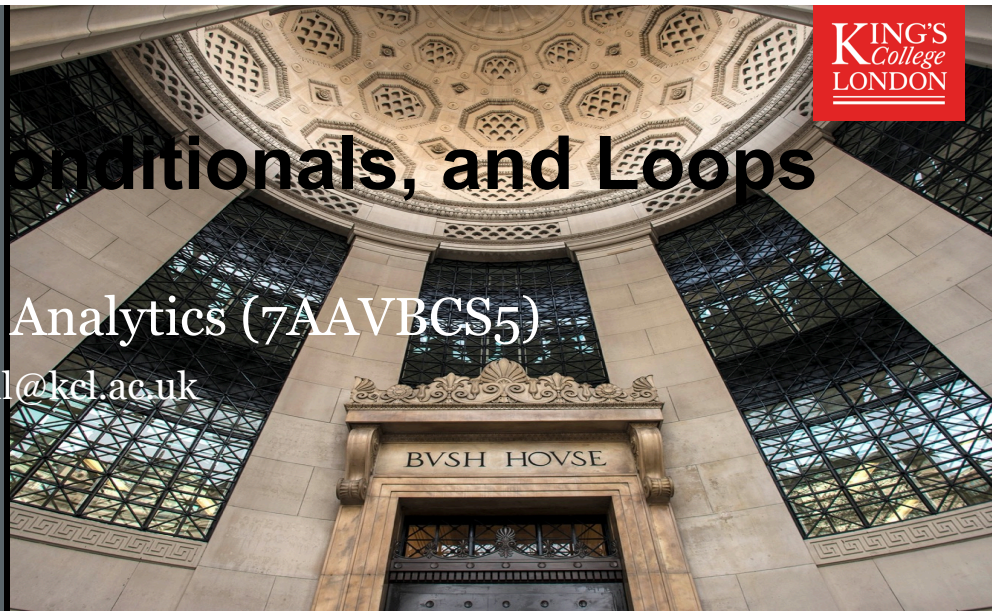


Functions, Conditionals, and Loops

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-11-23



Lecture Plan

- **Part I: Functions**

- Why functions?
- Function anatomy
- Built-in vs user-defined
- Best practices

- **Part II: Control Structures**

- `if` statements (revision)
- `for` loops
- `while` loops
- Combining concepts

- **Part III: Live coding**

- Key Learning Outcomes**

- By the end of this week you will be able to:

- Create and use custom functions in Python
- Use `for` and `while` loops for repetitive tasks
- Combine these for basic data-processing workflows

- Apply these tools to cultural analytics problems
- Work through a collection of **texts**, one at a time

Review: eight weeks without a loop

- Types, lists, dictionaries, arrays
- Comparisons and `if`
- Data frames: loading, selecting, filtering, missing data
- Categories, tables, crosstabs, `groupby`
- Bar plots, histograms, scatter plots, correlation

You have not needed a single loop yet.

That was deliberate. pandas does the repeating for you — `df["a"] * 2` applies to every row without being told to.

This week: what to do when it cannot.

Part I: Functions

Why functions?

“A function in R is something that takes a set of input values and returns an output value.” (Arnold & Tilton)

The same is true in Python.

Benefits

- Avoiding repetition in code
- Making code modular and readable

We have used a great many functions already — `len()`, `print()`, `round()`, `pd.read_csv()`.

Avoiding repetition

Write out a list by hand...

```
1 odd_numbers = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35,  
2               37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67, 69,  
3               71, 73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 99]
```



...or use `range()`:

```
1 odd_numbers = list(range(1, 100, 2))  
2 print(odd_numbers[:8], "...", len(odd_numbers), "numbers")
```

```
[1, 3, 5, 7, 9, 11, 13, 15] ... 50 numbers
```

Avoiding repetition, again

Use Python as a calculator...

```
1 average = sum(odd_numbers) / len(odd_numbers)
2 print(average)
```

```
50.0
```

...or use a function that already exists:

```
1 import statistics
2 print(statistics.mean(odd_numbers))
```

```
50
```

Function anatomy

```
1 round(3.14159, 2)
```

```
3.14
```

- **Function name:** `round` — what you are calling
- **Arguments:** `3.14159`, `2` — the inputs, inside the brackets
- **Return value:** `3.14` — what comes back

```
1 print(list(range(1, 10, 2)))
```

```
[1, 3, 5, 7, 9]
```

- **Parameter names:** `start`, `stop`, `step`. You need not name them, but doing so can make code more readable:
`range(start=1, stop=10, step=2)`

User-defined functions

```
1 def calculate_engagement(likes, retweets):
2     total = likes + retweets
3     return total
4
5
6 calculate_engagement(100, 50)
```

150

Component	R	Python
Keyword	function()	def
Body	inside { }	indented
Give back a value	return()	return

The trap: forgetting to return

```
1 def add_bad(a, b):
2     a + b          # calculated, then thrown away
3
4
5 def add_good(a, b):
6     return a + b
7
8
9 print("without return:", add_bad(2, 3))
10 print("with return    :", add_good(2, 3))
```

```
without return: None
with return    : 5
```

A function with no `return` gives you `None`. It does not error, and `None` will break something later, far from the cause.

Print is not return

```
1 def get_tax(salary, rate):
2     return salary * rate / 100
3
4
5 def print_tax(salary, rate):
6     print("My tax is", salary * rate / 100)
7
8
9 my_tax = get_tax(70000, 35)
10 net = 70000 - my_tax           # works, because get_tax returned a number
11 print("net income:", net)
12
13 print_tax(70000, 35)           # prints, but gives nothing back
```

```
net income: 45500.0
My tax is 24500.0
```

Parameters are local

```
1 def multiply_by_2(x):  
2     return x * 2  
3  
4  
5 a = 10  
6 print(multiply_by_2(a))  
7 print(multiply_by_2(5))
```

```
20  
10
```

The name `x` exists **only inside** the function. Many beginners think `x` must be defined outside first. It must not.

Best practices

Descriptive naming

```
1 # bad
2 def calc(x, y):
3     return x + y
4
5 # good
6 def calculate_total_engagement(likes, retweets):
7     return likes + retweets
```

Single responsibility

```
1 def clean_text(text): ...
2 def count_words(text): ...
3 def calculate_sentiment(text): ...
```

Documentation

```
1 def engagement_rate(likes, retweets, followers):
2     """Engagement as a percentage of followers"""
3
4     Returns a number between 0 and 100
5     """
6     total = likes + retweets
7     return (total / followers) * 100
```

Python uses a **docstring** — a string just inside the function — where R used `#` comments.

`help(engagement_rate)` then shows it.

Part II: Control Structures

Conditionals: a reminder

Conditionals let code make decisions based on whether something is `True` or `False`.

- `if` — run this only if the condition is true
- `elif` — test another condition if the previous was false
- `else` — run this if all previous were false

Operator	Meaning
<code><</code>	less than
<code>></code>	greater than
<code><=</code>	less than or equal
<code>>=</code>	greater than or equal
<code>==</code>	is equal to
<code>!=</code>	is not equal to

```
1 temperature = 30
2
3 if temperature > 25:
4     print("It's hot!")
```

```
5 elif temperature < 15:  
6     print("It's chilly!")  
7 else:  
8     print("It's warm.")
```

It's hot!

For loops

For loops repeat code once for each item in a sequence.

```
1 for i in range(1, 6):  
2     print(i)
```

```
1  
2  
3  
4  
5
```

```
1 cities = ["London", "Manchester", "Rome", "Lima"]  
2 for city in cities:  
3     print(f"I want to go to {city}")
```

```
I want to go to London  
I want to go to Manchester  
I want to go to Rome  
I want to go to Lima
```

When to use: when you know what you are iterating over — a list, a sequence, a set of files.

The cultural analytics example

```
1 from pathlib import Path
2
3 folder = Path(".././data")
4 for path in sorted(folder.glob("*.csv"))[:4]:
5     size_kb = path.stat().st_size / 1024
6     print(f"{path.name:50} {size_kb:8,.0f} KB")
```

bananas_apples.csv	0 KB
BritishAndIrishNewspapersTitleList_20191118.csv	8,793 KB
DawtryEtAl2015.csv	39 KB
fruitData.csv	0 KB

Collecting results, not just printing

```
1 lengths = []
2 for city in cities:
3     lengths.append(len(city))
4
5 print(lengths)
```

```
[6, 10, 4, 4]
```

Printing inside a loop shows you the answer.

Appending to a list keeps it. Beginners print when they meant to collect.

While loops

While loops repeat as long as a condition stays true. You do not know in advance how many times.

```
1 countdown = 5
2 while countdown > 0:
3     print(countdown)
4     countdown = countdown - 1
5 print("Lift off")
```

```
5
4
3
2
1
Lift off
```

You must change the condition inside the loop, or it runs forever. In a notebook, that means pressing the stop button.

Combining concepts

```
1 years = [1800, 1900, 1950, 2000, 2020]
2
3 modern = []
4 for year in years:
5     if year > 1900:
6         modern.append(year)
7
8 print(modern)
```

```
[1950, 2000, 2020]
```

A loop, a condition, and a collection — the three pieces together.

Loops and pandas

You still do not need a loop for a data frame.

```
1 import pandas as pd
2
3 museum = pd.read_csv("../data/museum_data.csv")
4 museum["expensive"] = museum["valuation"] > 20000 # no loop
5 print(museum["expensive"].sum(), "objects over 20,000")
```

```
100 objects over 20,000
```

Loops are for things pandas cannot vectorise — **files, documents, API calls, one-off objects.**

Part III: Live Coding

Exercise 1: a user-defined function

Write a function called `calculate_percentage` that takes two arguments, `part` and `total`, works out what percentage `part` is of `total`, and **returns** the result.

```
1 calculate_percentage(25, 100) # should give something meaning 25%
```

Exercise 2: an if statement

Write code that checks a variable called `grade` and prints:

- `"Pass"` if the grade is 50 or above
- `"Fail"` if it is below 50

Exercise 3: a for loop

You have a list of follower counts:

```
1 followers = [1000, 2500, 500, 3200, 1500]
```

Write a loop that goes through each value and prints it.

Then: collect the values into a new list instead of printing them.

Exercise 4: a while loop

Write a while loop that starts with `countdown = 5` and counts down to 1, printing each number, stopping when it reaches 0.

Exercise 5: putting it together

Read every `.txt` file in the data folder, and for each one print the filename and how many characters it contains. Then collect those counts into a **data frame** with one row per file.

Before next week

1. Complete `w09_worksheet.ipynb` in the workshop
2. Next week we start on **text**, and you will need loops

Note

The quiz in next week's lecture covers **this week's** material.

Debugging with AI

Loops are where AI assistance becomes genuinely tempting, because a loop that does nothing gives no error at all.

Ask <https://ai.create.kcl.ac.uk/chat> to **explain what your loop does step by step**, not to write one. If you cannot say what the loop variable holds on each pass, you will not be able to fix it in a closed quiz.



Speaker notes